Lesson 7: Eloquent Relationships

Contents

Introduction to Relationships

What are Relationships?

Relationships in Eloquent allow you to link Models together in an easy and clear way.

```
┌──────────────────────────────────────┐
│  User ↔ Posts (One to Many)          │
│  User ↔ Profile (One to One)         │
│  Post ↔ Tags (Many to Many)          │
└──────────────────────────────────────┘
```

Basic Relationship Types

Type	Description	Example
One to One	One-to-one relationship	User ← Profile
One to Many	One-to-many relationship	User ← Posts
Many to Many	Many-to-many relationship	Posts ←→ Tags

Type	Description	Example
Has One Through	One through another table	Country → User → Phone
Has Many Through	Many through another table	Country → Users → Posts
Polymorphic	Multi-type relationship	Comments on Posts/Videos

One to One

Concept

One-to-one relationship - each record in a table relates to only one record in another table.

Example: One user has one profile

```
User (id=1)  ↔  Profile (user_id=1)
User (id=2)  ↔  Profile (user_id=2)
```

Setup

Migration: users

```php
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamps();
});
```

Migration: profiles

```php
Schema::create('profiles', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('bio')->nullable();
    $table->string('avatar')->nullable();
    $table->string('website')->nullable();
    $table->timestamps();
```

```
});
```

Defining the Relationship

Model: User.php

```
class User extends Model
{
    public function profile()
    {
        return $this->hasOne(Profile::class);
    }
}
```

Model: Profile.php

```
class Profile extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Usage

```
// Get user's profile
$user = User::find(1);
$profile = $user->profile;

echo $profile->bio;
echo $profile->avatar;

// Get user from profile
$profile = Profile::find(1);
$user = $profile->user;

echo $user->name;
echo $user->email;

// Create profile for user
$user = User::find(1);
$profile = $user->profile()->create([
    'bio' => 'Laravel Developer',
```

```
    'website' => 'https://example.com',
]);

// Or
$profile = new Profile([
    'bio' => 'Laravel Developer',
    'website' => 'https://example.com',
]);
$user->profile()->save($profile);

// Check if relationship exists
if ($user->profile) {
    echo "Has profile";
}
```

One to Many

Concept

One-to-many relationship - each record in a table relates to multiple records in another table.

Example: One user has multiple posts

```
User (id=1)  ←  Post (user_id=1)
             ←  Post (user_id=1)
             ←  Post (user_id=1)
```

Setup

Migration: posts

```
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('title');
    $table->text('content');
    $table->timestamps();
});
```

Defining the Relationship

Model: User.php

```
class User extends Model
{
    public function posts()
    {
        return $this->hasMany(Post::class);
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Usage

```
// Get all user's posts
$user = User::find(1);
$posts = $user->posts;

foreach ($posts as $post) {
    echo $post->title;
}

// With Query Builder
$posts = $user->posts()->where('published', true)->get();
$posts = $user->posts()->latest()->take(5)->get();

// Count posts
$count = $user->posts()->count();

// Get post owner
$post = Post::find(1);
$user = $post->user;

echo $user->name;
```

```php
// Create post for user
$user = User::find(1);
$post = $user->posts()->create([
    'title' => 'My First Post',
    'content' => 'Content here...',
]);

// Or
$post = new Post([
    'title' => 'My First Post',
    'content' => 'Content here...',
]);
$user->posts()->save($post);

// Create multiple posts
$user->posts()->createMany([
    ['title' => 'Post 1', 'content' => 'Content 1'],
    ['title' => 'Post 2', 'content' => 'Content 2'],
]);
```

Many to Many

Concept

Many-to-many relationship - each record in a table relates to multiple records in another table and vice versa.

Example: A post has multiple tags, and a tag is used in multiple posts

```
Post (id=1)  ↔  post_tag  ↔  Tag (id=1)
Post (id=2)  ↔  post_tag  ↔  Tag (id=2)
```

Setup

Migration: tags

```php
Schema::create('tags', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->string('slug')->unique();
    $table->timestamps();
```

```
});
```

Migration: post_tag (Pivot Table)

```
Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->timestamps();

    $table->unique(['post_id', 'tag_id']);
});
```

Defining the Relationship

Model: Post.php

```
class Post extends Model
{
    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

Model: Tag.php

```
class Tag extends Model
{
    public function posts()
    {
        return $this->belongsToMany(Post::class);
    }
}
```

Usage

```
// Get all post tags
$post = Post::find(1);
$tags = $post->tags;

foreach ($tags as $tag) {
    echo $tag->name;
```

```
}

// Get all tag posts
$tag = Tag::find(1);
$posts = $tag->posts;

// Add tag to post
$post = Post::find(1);
$post->tags()->attach(1); // tag_id = 1
$post->tags()->attach([1, 2, 3]); // Multiple tags

// Remove tag
$post->tags()->detach(1);
$post->tags()->detach([1, 2]); // Multiple tags
$post->tags()->detach(); // Remove all

// Sync tags (replace)
$post->tags()->sync([1, 2, 3]);

// Sync without detaching
$post->tags()->syncWithoutDetaching([4, 5]);

// Toggle
$post->tags()->toggle([1, 2]); // Add or remove

// Check if relationship exists
if ($post->tags->contains($tag)) {
    echo "Has tag";
}
```

Pivot Table with Additional Data

Migration: post_tag

```
Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->integer('order')->default(0);
    $table->timestamps();
});
```

Model: Post.php

```
class Post extends Model
```

```
{
    public function tags()
    {
        return $this->belongsToMany(Tag::class)
                    ->withPivot('order')
                    ->withTimestamps();
    }
}
```

Usage:

```
// Attach with additional data
$post->tags()->attach(1, ['order' => 1]);

// Access additional data
foreach ($post->tags as $tag) {
    echo $tag->name;
    echo $tag->pivot->order;
    echo $tag->pivot->created_at;
}

// Sync with additional data
$post->tags()->sync([
    1 => ['order' => 1],
    2 => ['order' => 2],
    3 => ['order' => 3],
]);
```

Has One Through

Concept

One-through relationship - accessing a relationship through an intermediate table.

Example: Country → User → Phone

```
Country (id=1)  →  User (country_id=1)  →  Phone (user_id=1)
```

Setup

Migrations:

```
// countries
Schema::create('countries', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->timestamps();
});

// users
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->foreignId('country_id')->constrained();
    $table->string('name');
    $table->timestamps();
});

// phones
Schema::create('phones', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->string('number');
    $table->timestamps();
});
```

Defining the Relationship

Model: Country.php

```
class Country extends Model
{
    public function phone()
    {
        return $this->hasOneThrough(
            Phone::class,  // Final model
            User::class,   // Intermediate model
            'country_id',  // Foreign key on users
            'user_id',     // Foreign key on phones
            'id',          // Local key on countries
            'id'           // Local key on users
        );
    }
}
```

Usage

```php
$country = Country::find(1);
$phone = $country->phone;

echo $phone->number;
```

Has Many Through

Concept

Many-through relationship - getting multiple records through an intermediate table.

Example: Country → Users → Posts

```
Country (id=1)  →  Users (country_id=1)  →  Posts (user_id=...)
```

Defining the Relationship

Model: Country.php

```php
class Country extends Model
{
    public function posts()
    {
        return $this->hasManyThrough(
            Post::class,   // Final model
            User::class,   // Intermediate model
            'country_id',  // Foreign key on users
            'user_id',     // Foreign key on posts
            'id',          // Local key on countries
            'id'           // Local key on users
        );
    }
}
```

Usage

```php
// Get all posts from country users
$country = Country::find(1);
$posts = $country->posts;
```

```php
foreach ($posts as $post) {
    echo $post->title;
}

// With Query Builder
$posts = $country->posts()->where('published', true)->get();
$count = $country->posts()->count();
```

Polymorphic Relations

Concept

Polymorphic relationship - one model belongs to multiple other models.

Example: Comments on posts and videos

```
Post (id=1)    →  Comment (commentable_type='Post', commentable_id=1)
Video (id=1)   →  Comment (commentable_type='Video', commentable_id=1)
```

Setup

Migration: comments

```php
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->text('content');
    $table->morphs('commentable'); // Adds commentable_id and commentable_type
    $table->timestamps();
});

// Or manually:
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->text('content');
    $table->unsignedBigInteger('commentable_id');
    $table->string('commentable_type');
    $table->timestamps();

    $table->index(['commentable_id', 'commentable_type']);
});
```

Defining the Relationship

Model: Comment.php

```
class Comment extends Model
{
    public function commentable()
    {
        return $this->morphTo();
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}
```

Model: Video.php

```
class Video extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}
```

Usage

```
// Get post comments
$post = Post::find(1);
$comments = $post->comments;

// Get video comments
$video = Video::find(1);
$comments = $video->comments;

// Get item from comment
```

```php
$comment = Comment::find(1);
$commentable = $comment->commentable;

if ($commentable instanceof Post) {
    echo "Comment on post: " . $commentable->title;
} elseif ($commentable instanceof Video) {
    echo "Comment on video: " . $commentable->title;
}

// Create comment
$post = Post::find(1);
$comment = $post->comments()->create([
    'content' => 'Great post!',
]);

$video = Video::find(1);
$comment = $video->comments()->create([
    'content' => 'Nice video!',
]);
```

Many to Many Polymorphic

Concept

Many-to-many polymorphic relationship - complex relationship combining Many to Many and Polymorphic.

Example: Tags on posts and videos

```
Post (id=1)   ↔  taggables  ↔  Tag (id=1)
Video (id=1)  ↔  taggables  ↔  Tag (id=1)
```

Setup

Migration: taggables

```php
Schema::create('taggables', function (Blueprint $table) {
    $table->id();
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->morphs('taggable'); // taggable_id and taggable_type
    $table->timestamps();
```

```
});
```

Defining the Relationship

Model: Tag.php

```
class Tag extends Model
{
    public function posts()
    {
        return $this->morphedByMany(Post::class, 'taggable');
    }

    public function videos()
    {
        return $this->morphedByMany(Video::class, 'taggable');
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

Model: Video.php

```
class Video extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

Usage

```
// Add tags to post
$post = Post::find(1);
```

```php
$post->tags()->attach([1, 2, 3]);

// Add tags to video
$video = Video::find(1);
$video->tags()->attach([1, 2, 3]);

// Get all tag posts
$tag = Tag::find(1);
$posts = $tag->posts;
$videos = $tag->videos;

// Sync
$post->tags()->sync([1, 2, 3]);
```

Eager Loading

The Problem: N+1 Query

```php
// Bad: causes N+1 queries
$posts = Post::all(); // 1 query

foreach ($posts as $post) {
    echo $post->user->name; // N queries (one per post)
}
// Total: 1 + N queries
```

The Solution: Eager Loading

```php
// Good: only 2 queries
$posts = Post::with('user')->get(); // Only 2 queries

foreach ($posts as $post) {
    echo $post->user->name; // No additional queries
}
```

Eager Loading Examples

```php
// Load single relationship
$posts = Post::with('user')->get();
```

```php
// Load multiple relationships
$posts = Post::with(['user', 'comments'])->get();

// Load nested relationships
$posts = Post::with('user.profile')->get();
$posts = Post::with('comments.user')->get();

// Load multiple nested relationships
$posts = Post::with([
    'user',
    'comments.user',
    'tags'
])->get();

// Load with constraints
$posts = Post::with(['comments' => function ($query) {
    $query->where('approved', true)
          ->orderBy('created_at', 'desc');
}])->get();

// Load only specific number
$posts = Post::with(['comments' => function ($query) {
    $query->latest()->take(5);
}])->get();

// Lazy Eager Loading
$posts = Post::all();
$posts->load('user'); // Load after retrieval

// Load if not already loaded
$posts->loadMissing('user');
```

withCount - Count Relationships

```php
// Count comments
$posts = Post::withCount('comments')->get();

foreach ($posts as $post) {
    echo $post->comments_count; // Number of comments
}

// Count multiple relationships
$posts = Post::withCount(['comments', 'likes'])->get();

// Count with constraints
$posts = Post::withCount([
```

```
    'comments',
    'comments as approved_comments_count' => function ($query) {
        $query->where('approved', true);
    }
])->get();
```

Practical Examples

Example 1: Complete Blog System

```
// User and profile
$user = User::with('profile')->find(1);
echo $user->name;
echo $user->profile->bio;

// User posts with comments
$user = User::with('posts.comments')->find(1);

foreach ($user->posts as $post) {
    echo $post->title;
    echo "Comments: " . $post->comments->count();
}

// Post with owner, comments, and tags
$post = Post::with(['user.profile', 'comments.user', 'tags'])
           ->find(1);

echo $post->title;
echo "By: " . $post->user->name;
echo "Bio: " . $post->user->profile->bio;

foreach ($post->comments as $comment) {
    echo $comment->user->name . ": " . $comment->content;
}

foreach ($post->tags as $tag) {
    echo $tag->name;
}
```

Example 2: Statistics

```
// Users with post and comment counts
$users = User::withCount(['posts', 'comments'])
            ->orderBy('posts_count', 'desc')
            ->get();

foreach ($users as $user) {
    echo "{$user->name}: {$user->posts_count} posts, {$user->comments_count}
comments";
}

// Posts with approved comments count
$posts = Post::withCount([
    'comments as approved_comments_count' => function ($query) {
        $query->where('approved', true);
    }
])->get();
```

Example 3: Searching Relationships

```
// Posts from users in specific country
$posts = Post::whereHas('user', function ($query) {
    $query->where('country_id', 1);
})->get();

// Posts with comments from specific user
$posts = Post::whereHas('comments', function ($query) {
    $query->where('user_id', 1);
})->get();

// Posts without comments
$posts = Post::doesntHave('comments')->get();

// Posts with at least 5 comments
$posts = Post::has('comments', '>=', 5)->get();
```

Important Tips

Best Practices

1. **Use Eager Loading:**

```
// ✅ Good
$posts = Post::with('user')->get();
```

// ❌ Bad (N+1 problem) $posts = Post::all(); foreach ($posts as $post) { echo $post->user->name; }

````
2. **Use withCount for statistics:**
```php
// ✅ Good
$posts = Post::withCount('comments')->get();

// ❌ Bad
$posts = Post::all();
foreach ($posts as $post) {
    $count = $post->comments()->count(); // Query per post
}
````

3. **Specify required columns:**

```
$posts = Post::with('user:id,name,email')->get();
```

4. **Use whereHas instead of join:**

```
// ✅ Clearer and easier
$posts = Post::whereHas('tags', function ($query) {
$query->where('name', 'Laravel');
})->get();
```

⚠ Common Mistakes

1. **Forgetting Eager Loading:**

```
// ❌ N+1 problem
$posts = Post::all();
foreach ($posts as $post) {
echo $post->user->name; // Query per post
}
```
```

2. **Using get() twice:**

```
// ❌ Error
$posts = Post::with('user')->get()->get();
```

// ✅ Correct $posts = Post::with('user')->get();

```
3. **Forgetting to define belongsTo:**
```php
// ❌ Only hasMany
class User extends Model {
    public function posts() {
        return $this->hasMany(Post::class);
    }
}

// ✅ With belongsTo in Post
class Post extends Model {
    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

Next Step

After completing this lesson, you're ready for:

Lesson 8: Validation and Form Requests

- Basic Validation
- Form Request Classes
- Custom Validation Rules
- Error Messages

Happy Learning! ✨
```